Credit Lecture 3: Deep Learning, an Overview

Deep Learning for Actuarial Modelling: a credit risk companion

AUTHOR

Mario Ervedosa, adapting Scognamiglio, Maggi and Wüthrich

PUBLISHED

September 1, 2026

ABSTRACT

We stand back from the fitted models of the first two credit lectures and ask what deep learning is for. Lecture 3 of Deep Learning for Actuarial Modeling answers with representation learning, i.e. the actuary supplies raw data and the architecture learns what to extract, and this companion transplants that answer to credit risk. The authors' Fashion-MNIST comparison is replaced by a wide anonymised credit card portfolio, on which principal components, an autoencoder and a supervised feature extractor each compress 1,158 covariates to two dimensions. The autoencoder reconstructs the portfolio far better than the principal components and discriminates default far worse, which is the result the rest of the series is built to avoid. The lecture closes on the architectures that follow, and on the combined actuarial neural network, whose GLM baseline is the scorecard a credit committee has already approved.

Introduction

The first two credit lectures built a probability-of-default model the classical way. The first chose covariates by inspection, banded borrower age by hand, and read an interaction off a Simpson's paradox in declared income. The second supplied the distributional theory underneath those choices, and in doing so it also exposed their

cost, since the seven age bands had to be merged to six before maximum likelihood would behave. Every one of those decisions was taken by a person looking at a plot.

This lecture stands back and asks what deep learning offers instead. The answer the course gives is **representation learning**: rather than hand-crafting the features a regression consumes, the actuary hands raw data to an architecture and lets the training algorithm decide what to extract. Lecture 3 of the course develops that idea, builds the feed-forward network as a generalisation of the GLM, surveys the architectures that specialise it to sequences and images, and closes on the combined actuarial neural network (CANN), which adds a learned correction to a fitted GLM baseline. This companion follows the same arc for credit.

Two things here are ours rather than the authors'. The first is the register: an overview lecture delivered as slides is a sequence of claims, and a document has to argue for them, so the material is recast as prose. The second is the evidence. The authors motivate representation learning with Fashion-MNIST, where a deep autoencoder separates ten balanced clothing classes that principal components leave smeared together. Credit has no Fashion-MNIST, and it does have wide anonymised portfolios, so we run the same comparison on 1,214 unlabelled features from a credit card book and report what happens. The result does not flatter the autoencoder, and it turns out to be the most useful thing in the lecture.

1 Why deep learning in credit risk?

1.1 The scorecard and its hand-crafted features

Credit scoring has a mature and highly disciplined feature engineering tradition, which is precisely why it is the right place to see what representation learning replaces. The classical scorecard takes each candidate variable, bins it into coarse classes, replaces each class by its weight of evidence, and admits the variable only if the binning is monotone,

populated at every level and stable across time. The procedure is transparent, auditable and slow, and every step of it is a decision recorded in a model development document.

The second credit lecture ran a miniature version of that procedure without naming it. Borrower age was cut into seven bands at boundaries chosen by eye from a default-rate plot. The oldest band held thirteen loans and no defaults, which drove its coefficient towards minus infinity, so the two oldest bands were merged into `61+`. Missing home ownership and missing employment duration were assigned an explicit `missing` level, on the judgement that an applicant who skips a field differs from one who fills it in. Each decision was defensible on its own, and together they are the model's feature engineering. A network would have made none of them.

1.2 The curse of dimensionality arrives as a column count

Hand-classing works while the covariate count is small enough for a person to look at every variable. Credit data left that regime some time ago.

The credit card portfolio this lecture uses carries 1,214 anonymised features per account, arranged in four blocks: 48 on-us attributes, 664 transaction attributes, 452 bureau attributes and 50 bureau enquiry attributes. Two of the bureau columns arrive as strings and are empty for every account, so 1,212 columns are numeric and enter the design matrix below. Nobody classed 1,214 variables by hand, and nobody will. The transaction block alone is almost certainly a set of aggregates computed over a customer-month panel before the file was released, which is to say somebody's feature engineering has already happened and has been frozen into the data; that reading is an inference from the shape of the columns rather than a documented fact, and it should be hedged accordingly. Meanwhile a telematics feed, a text field, a bureau timeline or a transaction stream arrives with no natural finite covariate list at all.

Consequently the actuarial question shifts. Instead of asking which transformation of which variable to admit, we ask what the architecture should be allowed to learn, and how to tell whether what it learned is worth having.

1.3 Historical perspective

The course sets out the machine learning strand of the history, and we take it as theirs: the McCulloch-Pitts neuron in 1943, Rosenblatt's perceptron in 1958, the first AI winter after Minsky and Papert in 1969, the popularisation of back-propagation by Rumelhart, Hinton and Williams in 1986, the universal approximation theorems of Cybenko and Hornik between 1989 and 1991, long short-term memory and LeNet-5 in the late 1990s, AlexNet's 2012 breakthrough in computer vision, sequence-to-sequence models and residual networks between 2014 and 2016, the Transformer in 2017, and foundation models from 2020. Their scaling-laws slide makes the point that compute, data and depth improve performance together and predictably.

Credit risk has its own strand, and it is older than the actuarial one. Discriminant analysis on financial ratios dates to Altman's 1968 Z-score, and logistic regression entered consumer credit scoring in the late 1970s and early 1980s, which is why the logistic GLM of the second lecture is the industry's incumbent rather than one option among many. Basel II made the internal ratings based approach a capital methodology in 2004, and in doing so it attached a regulatory apparatus to the PD model that no insurance pricing model carries. Machine learning classifiers were benchmarked against scorecards by Baesens and co-authors in 2003 and again by Lessmann and co-authors in 2015, with the consistent finding that the gains are real and modest. Deep learning arrived later here than in vision or language, and the reason is not technical.

1.4 What the regulator asks in return

The reason is that a credit decision is a decision about a person, and it has to be explainable to that person. A declined applicant is entitled to know why. A supervisor reviewing an IRB model expects each risk driver to have an economic rationale, a stable relationship to default, and a documented rank ordering. Under the EU AI Act, creditworthiness assessment of natural persons sits in Annex III among the high-risk uses, with obligations on data governance, documentation and human oversight attached. The European Banking Authority's 2021 discussion paper on machine learning for IRB models, and its 2023 follow-up report, take the same line from the prudential side: the technique is not prohibited, and the burden of explanation rises with its complexity.

Hence the architectures this course spends most of its time on are exactly the ones a credit modeller needs. LocalGLMnet learns a GLM whose coefficients vary with the covariates, so every prediction decomposes into per-variable contributions that read like a scorecard's points. The CANN keeps the approved GLM as its baseline and learns only a correction to it. Both are attempts to buy the flexibility of a network without surrendering the account of itself that a credit model has to give.

2 Representation learning: the core principle

2.1 The idea, stated formally

Representation learning is the machine learning technique in which the algorithm designs the features. Given covariates $\boldsymbol{X} \in \mathbb{R}^q$ and a response D, we seek a mapping

$$\phi : \mathbb{R}^q \to \mathbb{R}^p, \qquad \boldsymbol{Z} = \phi(\boldsymbol{X}), \qquad p \ll q,$$

such that simple predictors acting on $\boldsymbol{Z}$ perform well (Bengio, Courville and Vincent, 2013). The classical members of this family are principal component analysis and partial least squares, both linear. Deep learning is the version in which ϕ is a composition of many non-linear layers, so the features of one layer are built from the features of the layer below, and the hierarchy is what lets a modest parameter count express an intricate transformation.

PRINCIPAL COMPONENTS, PARTIAL LEAST SQUARES AND THE REST OF THE LINEAR FAMILY

Both classical mappings fix ϕ to a linear one, $\boldsymbol{Z}_m = \boldsymbol{X}\alpha_m$, and they differ only in the criterion that selects the directions α_m. Since the demonstration later in this lecture turns on exactly that choice of criterion, it is worth setting the two out properly. Throughout this callout $\boldsymbol{X}$ denotes the $n \times q$ design matrix and S its sample covariance matrix, in place of the single covariate vector the surrounding text uses. The number of retained directions is written M, playing the part that p plays in the definition above.

Principal components. PCA is unsupervised, and the response plays no part in it. On standardised covariates the mth direction solves

$$\max_{\alpha} \ \mathrm{Var}(\boldsymbol{X}\alpha)$$
$$\text{subject to} \quad \|\alpha\| = 1, \quad \alpha^\top S v_\ell = 0, \quad \ell = 1, \dots, m-1,$$

so each direction claims the most variance left once its predecessors are accounted for, and the constraint $\alpha^\top S v_\ell = 0$ makes the resulting scores mutually uncorrelated (Hastie, Tibshirani and Friedman, 2009, equation 3.63). The solutions are the leading eigenvectors of S, equivalently the right singular vectors of the centred design matrix. Deisenroth, Faisal and Ong (2020) derive the same subspace twice over, once as the maximiser of retained variance and once as the minimiser of squared reconstruction error, so a component admits two readings, as a direction of maximal spread and as part of the best low-rank summary of the same data. That second reading is why the linear compression is the benchmark the autoencoder below has to beat on its own terms. Note also that the criterion depends on the units, which is why the covariates are standardised first.

Principal components regression then regresses the response on the first M scores. Because the full set of q scores spans the same column space as $\boldsymbol{X}$, taking $M = q$ returns the ordinary least squares fit exactly. Hence truncating at $M < q$ acts as a shrinkage device on the same model rather than fitting a different one.

Partial least squares. PLS brings the response into the construction. After standardising, it sets $\hat{\varphi}_{1j} = \langle x_j, y \rangle$ and forms the first direction $z_1 = \sum_j \hat{\varphi}_{1j} x_j$, so

every covariate is weighted by the strength of its univariate effect on the outcome. The response is regressed on z_1, every covariate is then orthogonalised with respect to z_1, and the step repeats on the residualised covariates until $M \leq q$ directions are in hand (Hastie, Tibshirani and Friedman, 2009, algorithm 3.3). The criterion behind that algorithm is

$$\max_{\alpha} \ \operatorname{Corr}^2(y, \boldsymbol{X}\alpha)\operatorname{Var}(\boldsymbol{X}\alpha)$$
$$\text{subject to} \quad \|\alpha\| = 1, \quad \alpha^\top S\hat{\varphi}_\ell = 0, \quad \ell = 1, \dots, m-1,$$

established by Stone and Brooks (1990) and Frank and Friedman (1993), the algorithm itself being Wold's (1975). Whereas PCA keys on variance alone, PLS trades variance off against correlation with the response. In practice the variance factor tends to dominate the product, so PLS ends up behaving much like ridge regression and principal components regression, and $M = q$ again recovers ordinary least squares.

The rest of the family. Linear discriminant analysis is the supervised sibling that chooses the subspace maximising between-class scatter relative to within-class scatter, and it is the mapping behind Altman (1968), so credit has been doing supervised linear representation learning since well before the term existed. Non-linear embeddings such as t-SNE compress to two dimensions as well, and they serve as diagnostic plots, since the map is fitted to the sample in hand and offers no way to place a new applicant.

Why the criterion matters in credit. Consider IFRS 9 forward-looking modelling, where a decade of quarterly observations has to support a wide set of candidate macroeconomic indicators. PCA is the usual reduction there. However, the components are built without any reference to the default rate, so a component can load its dominant predictor in the direction that contradicts economic expectation, and the model built on it then carries a counter-intuitive sign (Djurovic, 2025). The practical remedy is to check the directional alignment of each raw predictor against the target after the reduction and before the component is accepted as an input. Alternatively, condition the composite on the outcome from the outset and impose the signs directly, which is what a supervised macroeconomic index does (Djurovic, 2026), and which is PLS's argument restated in a regulatory register.

> **Where principal components do earn their place.** A production scorecard build reaches for PCA away from the model input, in the variable reduction that precedes it. Correlated covariates are clustered by principal component, a cluster whose second eigenvalue exceeds one is split again, and the most predictive member of each surviving cluster is retained. The eigen-decomposition of the correlation matrix then returns as the condition index that flags severe multicollinearity among whatever survived, which is a diagnostic on the covariates rather than a transformation of them.
>
> The comparison later in this lecture puts these criteria to the test on the credit card book. Two principal components, an autoencoder of the same width, and a supervised extractor of the same width all pass through one readout, and the ranking they earn on reconstruction error reverses on discrimination.

Depth earns its place through that hierarchy. A shallow network can approximate a complicated function, and to do so it needs to be very wide, in the way a step function approximates a diagonal line only by taking many steps. Deeper layers instead compose: early layers detect simple structure and later layers combine it into abstractions, which is visible directly in the convolutional filters that Zeiler and Fergus (2014) visualised, where layer one finds edges and layer five finds objects.

The word “simple” in the definition above is doing quiet work, and the rest of this section is about it. A representation is good relative to a task, so a compression that is excellent for one purpose can be useless for another. The authors’ Fashion-MNIST example picks a case where the two purposes coincide. Credit, as we are about to see, is not so obliging.

2.2 The credit card portfolio

The dataset is an anonymised credit card book of 96,806 accounts, released as a development and validation pair, with a `bad_flag` on the development file at 1.42 per cent and no flag at all on the validation file. It is public, and it is published on Kaggle as Credit Card Behaviour Score; the two file names this repository carries are the uploader’s own. It is the opposite of Bondora in every respect that matters here. The

features carry no meaning a modeller could interpret, so there is no GLM story to tell about any coefficient, and the 1.42 per cent bad rate reproduces the rare-event regime of the course's claim counts rather than Bondora's 29 per cent. That combination, wide and anonymous and imbalanced, is exactly the regime in which hand-crafted features are impossible and automated ones are the only option on the table.

One structural feature of the file matters for what follows. A block of 626 columns, comprising every on-us and transaction attribute, is missing together for the same 25,231 accounts, or 26.1 per cent of the book. Those accounts have no on-us record and are described by bureau data alone. The missingness is therefore a fact about the account rather than a defect in the file, and a representation of this portfolio has to accommodate two populations described by different variables.

```
import time
import numpy as np
import polars as pl
import torch
import torch.nn as nn
import matplotlib.pyplot as plt
import statsmodels.api as sm
from sklearn.decomposition import PCA
from sklearn.metrics import roc_auc_score

dat = pl.read_parquet("../data/credit_card_dev.parquet")
feat = [c for c, t in zip(dat.columns, dat.dtypes)
        if t != pl.String and c not in ("account_number", "bad_flag")]
y = dat["bad_flag"].to_numpy().astype(np.float32)
X = dat.select(feat).to_numpy().astype(np.float32)
print(f"{dat.height:,} accounts, {len(feat):,} numeric features, "
      f"bad rate {y.mean():.4%}")
```

```
96,806 accounts, 1,212 numeric features, bad rate 1.4173%
```

2.3 Pre-processing, and one decision that turned out to matter

The design matrix is built in four steps, all fitted on the learning sample alone. Missing values are replaced by the column median, the columns are centred and scaled, the 54 columns that are constant on the learning sample are dropped, and the standardised values are clipped to ± 10 standard deviations.

The clipping is the step worth stating openly, because it is the one that decided whether the supervised network below trained at all. Several of these columns are monetary aggregates with extremely heavy right tails, and after standardisation a handful of accounts sit hundreds of standard deviations from the mean. Gradient descent on such a matrix takes enormous steps on the batches containing those rows, and in a diagnostic run without the clip, not reproduced in this document, the supervised network never recovered from them and finished at a test AUC of about 0.62, below the principal components it should have beaten. Clipping at ten standard deviations left the ranking of every account intact and lifted it to the 0.82 reported below. The course's own covariate pre-processing section says to standardise; on credit data with monetary tails, saying so is not quite enough.

```
rng = np.random.default_rng(1)
perm = rng.permutation(len(X))
n_learn = int(0.8 * len(X))
learn, test = perm[:n_learn], perm[n_learn:]

med = np.nan_to_num(np.nanmedian(X[learn], axis=0))
X = np.where(np.isnan(X), med, X)
mu, sd = X[learn].mean(0), X[learn].std(0)
keep = sd > 0
X = np.clip((X[:, keep] - mu[keep]) / sd[keep], -10,
        10).astype(np.float32)

print(f"design matrix {X.shape[0]:,} x {X.shape[1]:,} "
      f"(dropped {int((~keep).sum())} constant columns)")
print(f"learn {len(learn):,} accounts, {int(y[learn].sum()):,} bad "
      f"({y[learn].mean():.4%});  "
      f"test {len(test):,} accounts, {int(y[test].sum()):,} bad "
      f"({y[test].mean():.4%})")
```

```
design matrix 96,806 x 1,158 (dropped 54 constant columns)
learn 77,444 accounts, 1,113 bad (1.4372%);  test 19,362 accounts, 259
bad (1.3377%)
```

The split is random, since this file carries no origination date. The out-of-time contrast that dominated the second lecture is therefore unavailable here, and everything below should be read as the flattering case.

2.4 Three representations, two dimensions each

We compress 1,158 covariates to two dimensions three times over, and then judge each compression by the same standard. Two dimensions is a brutal bottleneck, chosen because it is the authors' choice and because it can be plotted; the point of the exercise is the comparison rather than the absolute level.

The three mappings differ only in what trains them. **Principal components** minimise linear reconstruction error and are the optimal linear compression at any given width. The **autoencoder** minimises non-linear reconstruction error, passing the covariates through $1158 \to 256 \to 64 \to 2$ and back out again, and it never sees the default flag.

The **supervised extractor** is a feed-forward network with a two-neuron penultimate layer, trained on the Bernoulli deviance with dropout and early stopping, so its two features are learned against the response.

Each representation is then scored the same way. A logistic GLM is fitted to the two codes on the learning sample and evaluated on the test sample, using the Bernoulli deviance in 10^{-2} units and the AUC, as the second lecture did. The GLM readout is the course's own architecture read backwards, since a network is a feature extractor composed with a GLM, and holding the readout fixed isolates the quality of the features.

```
def readout(Z, label):
    fit = sm.GLM(y[learn], sm.add_constant(Z[learn]),
                 family=sm.families.Binomial()).fit()
    p = np.clip(fit.predict(sm.add_constant(Z[test])), 1e-7, 1 - 1e-7)
    dev = 100 * 2 * np.mean(-y[test] * np.log(p) - (1 - y[test]) *
        np.log(1 - p))
    auc = roc_auc_score(y[test], p)
    print(f"{label:24s} AUC {auc:.3f}   deviance {dev:6.3f}")
    return auc

p0 = y[learn].mean()
dev0 = 100 * 2 * np.mean(-y[test] * np.log(p0) - (1 - y[test]) * np.log(1
        - p0))
print(f"{'null model':24s} AUC   -     deviance {dev0:6.3f}")
```

```
null model               AUC   -     deviance 14.207
```

```
pca = PCA(n_components=2, random_state=1).fit(X[learn])
Z_pca = pca.transform(X)
recon = pca.inverse_transform(Z_pca)
mse_pca = np.mean((X[test] - recon[test]) ** 2)
print(f"PCA: {pca.explained_variance_ratio_.sum():.1%} of variance
        retained, "
      f"test reconstruction MSE {mse_pca:.3f}")
auc_pca = readout(Z_pca, "PCA codes")
```

```
PCA: 11.5% of variance retained, test reconstruction MSE 0.524
PCA codes                AUC 0.737   deviance 13.510
```

The autoencoder trains on the learning sample for 60 epochs of Adam over mini-batches of 1,024 accounts, on the CPU with every seed fixed, so the numbers below reproduce on re-render.

```
def fit_autoencoder(seed=1, epochs=60, batch=1024):
    torch.manual_seed(seed)
    q = X.shape[1]
    net = nn.Sequential(
        nn.Linear(q, 256), nn.ReLU(), nn.Linear(256, 64), nn.ReLU(),
        nn.Linear(64, 2),                                  # the
        bottleneck
        nn.Linear(2, 64), nn.ReLU(), nn.Linear(64, 256), nn.ReLU(),
        nn.Linear(256, q))
    Xt = torch.tensor(X)
    opt, mse = torch.optim.Adam(net.parameters()), nn.MSELoss()
    gen = torch.Generator().manual_seed(seed)
    idx = torch.tensor(learn)
    for _ in range(epochs):
        order = idx[torch.randperm(len(idx), generator=gen)]
        for chunk in order.split(batch):
            opt.zero_grad()
            mse(net(Xt[chunk]), Xt[chunk]).backward()
            opt.step()
    net.eval()
    with torch.no_grad():
        held = float(np.mean([mse(net(Xt[c]), Xt[c]).item()
                              for c in torch.tensor(test).split(8192)]))
        Z = torch.cat([net[:5](Xt[i:i + 8192])
                       for i in range(0, len(Xt), 8192)]).numpy()
    return Z, held

t0 = time.time()
Z_ae, mse_ae = fit_autoencoder()
print(f"autoencoder: test reconstruction MSE {mse_ae:.3f}, "
      f"{1 - mse_ae / mse_pca:.0%} below PCA  [{time.time() - t0:.0f}s]")
auc_ae = readout(Z_ae, "autoencoder codes")
```

```
autoencoder: test reconstruction MSE 0.407, 22% below PCA  [91s]
autoencoder codes        AUC 0.644   deviance 14.052
```

The supervised extractor uses the same machinery the next lecture develops in full: a validation slice carved out of the learning sample, checkpointing on the validation

deviance, and early stopping once twenty epochs pass without improvement. Dropout at 0.3 is included because 1,113 bad accounts against 1,158 covariates is a regime in which a network memorises quickly.

```
def fit_supervised(seed=1, epochs=60, batch=1024, patience=20):
    torch.manual_seed(seed)
    encoder = nn.Sequential(
        nn.Linear(X.shape[1], 64), nn.ReLU(), nn.Dropout(0.3),
        nn.Linear(64, 16), nn.ReLU(), nn.Dropout(0.3),
        nn.Linear(16, 2))                                  # the
        bottleneck
    net = nn.Sequential(encoder, nn.Linear(2, 1))
    Xt, yt = torch.tensor(X), torch.tensor(y).unsqueeze(1)
    gen = torch.Generator().manual_seed(seed)
    cut = int(0.2 * len(learn))
    val, fit = torch.tensor(learn[:cut]), torch.tensor(learn[cut:])
    opt, bce = torch.optim.Adam(net.parameters(), lr=3e-4),
        nn.BCEWithLogitsLoss()
    best, best_epoch, state, hist = np.inf, 0, None, []
    for epoch in range(1, epochs + 1):
        net.train()
        for chunk in fit[torch.randperm(len(fit),
        generator=gen)].split(batch):
            opt.zero_grad()
            bce(net(Xt[chunk]), yt[chunk]).backward()
            opt.step()
        net.eval()
        with torch.no_grad():
            hist.append((200 * bce(net(Xt[fit]), yt[fit]).item(),
                         200 * bce(net(Xt[val]), yt[val]).item()))
        if hist[-1][1] < best:
            best, best_epoch = hist[-1][1], epoch
            state = {k: v.clone() for k, v in net.state_dict().items()}
        if epoch - best_epoch >= patience:
            break
    net.load_state_dict(state)
    net.eval()
    with torch.no_grad():
        Z = torch.cat([encoder(Xt[i:i + 8192])
                       for i in range(0, len(Xt), 8192)]).numpy()
    return Z, np.array(hist), best_epoch

t0 = time.time()
Z_sup, hist, stop = fit_supervised()
print(f"supervised extractor: validation optimum at epoch {stop} of "
      f"{len(hist)} run  [{time.time() - t0:.0f}s]")
auc_sup = readout(Z_sup, "supervised features")
print(f"correlation of the two supervised coordinates: "
      f"{np.corrcoef(Z_sup[:, 0], Z_sup[:, 1])[0, 1]:+.2f}")
```

```
supervised extractor: validation optimum at epoch 9 of 29 run  [21s]
supervised features      AUC 0.820   deviance 12.328
correlation of the two supervised coordinates: −0.97
```

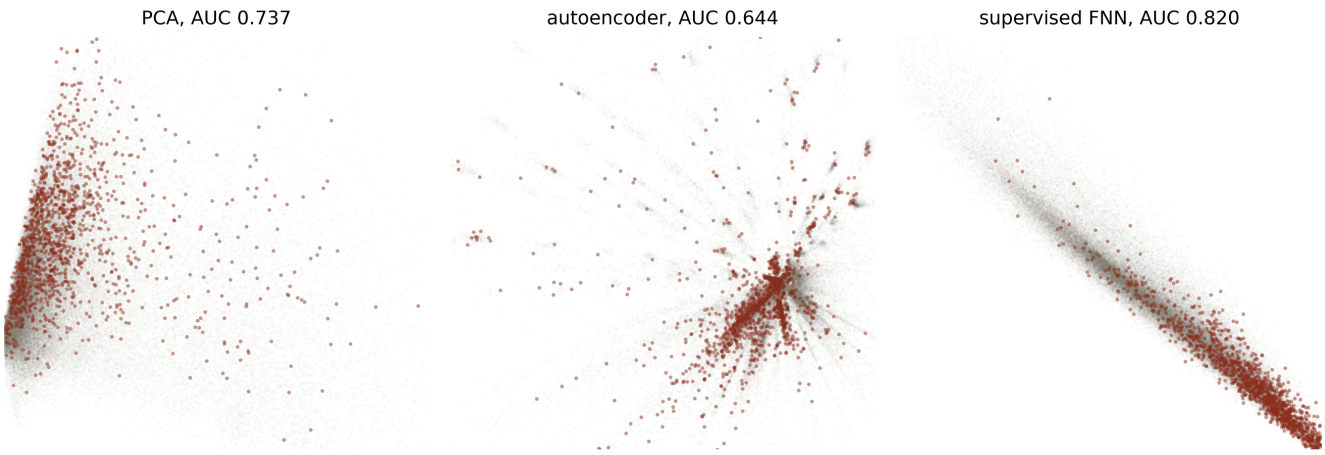
2.5 What the comparison shows

The autoencoder wins the contest it was entered for and loses the one that matters. Its test reconstruction error of 0.407 is 22 per cent below the 0.524 that two principal components achieve, so the non-linear compression does capture structure the optimal linear compression of the same width cannot reach. It is a better summary of the portfolio by its own standard. Then the readout reverses the ranking. A logistic GLM on the autoencoder's two codes scores an AUC of 0.644 against 0.737 for the principal components, and its test deviance of 14.052 sits barely below the null model's 14.207. Two dimensions of unsupervised non-linear compression have discarded almost all of the default signal in the book.

The supervised extractor, at the same two dimensions and through the same readout, reaches an AUC of 0.820 and a deviance of 12.328. The data are identical, the bottleneck width is identical, and the only thing that changed is the loss that trained the mapping.

```
panels = [(f"PCA, AUC {auc_pca:.3f}", Z_pca),
          (f"autoencoder, AUC {auc_ae:.3f}", Z_ae),
          (f"supervised FNN, AUC {auc_sup:.3f}", Z_sup)]
good, bad = y == 0, y == 1

fig, axs = plt.subplots(1, 3, figsize=(9, 3.2))
for ax, (title, Z) in zip(axs, panels):
    ax.scatter(Z[good, 0], Z[good, 1], s=1, c="#8A8377", alpha=0.03,
        lw=0)
    ax.scatter(Z[bad, 0], Z[bad, 1], s=3, c="#8C2F1E", alpha=0.5, lw=0)
    lo, hi = np.percentile(Z, 0.2, axis=0), np.percentile(Z, 99.8,
        axis=0)
    ax.set_xlim(lo[0], hi[0]); ax.set_ylim(lo[1], hi[1])
    ax.set_title(title, fontsize=9)
    ax.set_xticks([]); ax.set_yticks([])
    for spine in ax.spines.values():
        spine.set_visible(False)
plt.tight_layout(); plt.show()
```

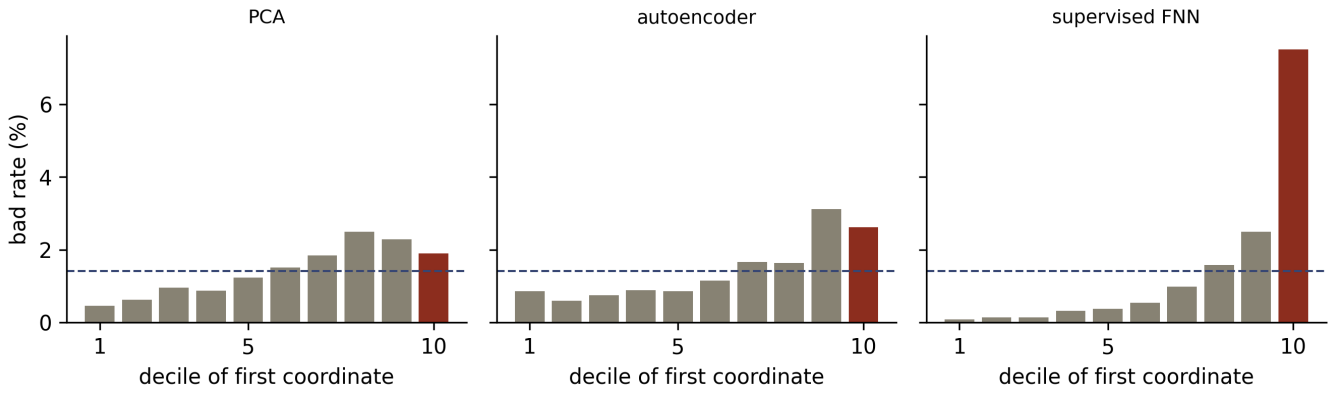


Each panel plots all 96,806 accounts in the two dimensions of one representation: 95,434 good accounts in grey and 1,372 bad accounts overplotted in red. Bad accounts are drawn on top and at a larger marker size, so the eye sees them out of proportion to their 1.42 per cent share of the book. Axes are unlabelled because the coordinates are arbitrary; the axis limits trim the outer 0.2 per cent of each representation so the bulk is visible. Panel scales are not comparable with one another.

The figure says the same thing without any statistic. Under the principal components the bad accounts crowd towards one edge of the cloud while remaining thoroughly mixed with the good ones. Under the autoencoder they scatter along the spikes of a star-shaped manifold with no region that is distinctively theirs, which is what an AUC of 0.644 looks like. Under the supervised extractor the whole book collapses onto a ridge and the bad accounts occupy one end of it.

That collapse is worth a remark of its own. The correlation between the supervised extractor's two coordinates is -0.97, so the network given two dimensions used approximately one. There is a single thing to predict and the deviance rewards nothing else, hence the extractor learns a scalar score and the second neuron is close to redundant. An unsupervised objective faces no such pressure and spreads itself over whatever the covariance structure offers.

```
fig, axs = plt.subplots(1, 3, figsize=(9, 2.8), sharey=True)
for ax, (title, Z) in zip(axs, panels):
    score = Z[:, 0] * np.sign(np.corrcoef(Z[:, 0], y)[0, 1])
    decile = np.searchsorted(np.quantile(score, np.linspace(0, 1, 11)
        [1:-1]), score)
    rate = np.array([100 * y[decile == k].mean() for k in range(10)])
    ax.bar(range(1, 11), rate, color="#8A8377")
    ax.bar([10], [rate[9]], color="#8C2F1E")
    ax.axhline(100 * y.mean(), color="#3B4A75", ls="--", lw=1)
    ax.set_title(title.split(",")[0], fontsize=9)
    ax.set_xticks([1, 5, 10]); ax.set_xlabel("decile of first
        coordinate")
    for spine in ("top", "right"):
        ax.spines[spine].set_visible(False)
axs[0].set_ylabel("bad rate (%)")
plt.tight_layout(); plt.show()
```



Observed bad rate by decile of each representation's first coordinate, on a shared vertical scale, with the portfolio's 1.42 per cent rate marked by the dashed line and the top decile in red. Each coordinate is oriented so that higher means riskier. The supervised representation ramps monotonically to 7.5 per cent in the top decile; the two unsupervised representations peak before their top decile and then fall back.

The decile view adds the detail an AUC hides. The supervised score ramps monotonically from 0.1 per cent in its safest decile to 7.5 per cent in its riskiest, a lift of

5.3 times the portfolio rate, which is the shape a credit committee expects of a scorecard. The PCA score peaks in its eighth decile and the autoencoder score in its ninth, and both then fall back in the tenth, so neither is monotone in default risk. A rating system built on either would need its grades reordered by hand before anyone could use it.

2.6 Why the autoencoder loses, measured rather than asserted

The mechanism is general, and here it can be checked. Reconstruction error is dominated by the directions carrying the most variance and by the blocks holding the most columns, whereas default is a rare event that need not live in either. The four column blocks let us test that directly, by measuring how much individual default signal each block carries per column against how much of the leading principal component's loading it attracts.

```
def block_of(name):
    if name.startswith("bureau_enquiry"): return "bureau enquiry"
    if name.startswith("bureau"): return "bureau"
    if name.startswith("onus"): return "on-us"
    return "transaction"

kept = np.array([block_of(f) for f, k in zip(feat, keep) if k])
signal = np.array([abs(roc_auc_score(y, X[:, j]) - 0.5)
                   for j in range(X.shape[1])])
loading = pca.components_[0] ** 2

for b in ["on-us", "transaction", "bureau", "bureau enquiry"]:
    m = kept == b
    print(f"{b:15s} {m.sum():4d} columns ({m.sum() / len(kept):5.1%})   "
          f"mean |AUC-0.5| {signal[m].mean():.4f}   "
          f"PC1 loading share {loading[m].sum():.3f}")
```

```
on-us             47 columns ( 4.1%)   mean |AUC-0.5| 0.0721   PC1
loading share 0.006
transaction      658 columns (56.8%)   mean |AUC-0.5| 0.0132   PC1
loading share 0.006
bureau           408 columns (35.2%)   mean |AUC-0.5| 0.0302   PC1
loading share 0.970
bureau enquiry    45 columns ( 3.9%)   mean |AUC-0.5| 0.0710   PC1
loading share 0.018
```

The numbers are stark. The two blocks carrying the most default signal per column are the 47 on-us attributes and the 45 bureau enquiry attributes, each averaging an individual AUC about 0.072 away from a coin flip. Between them they are 8 per cent of the matrix, and they attract 2.4 per cent of the leading principal component's loading. That component instead puts 97 per cent of its weight on the bureau block, which is middling per column and large. Meanwhile the transaction block is 56.8 per cent of the matrix and averages 0.013, so more than half the covariates carry almost no individual default signal at all while consuming more than half of any reconstruction budget.

Consequently an unsupervised representation of this portfolio is largely a representation of the bureau block, and default does not live there in proportion. What the bureau columns actually measure cannot be checked, since the file is anonymised, so treat any account of why they dominate the variance as inference rather than as a finding; the loading share itself is measured. The general result stands whatever they turn out to be.

Wherever the loudest variation in a dataset is not the variation the response cares about, reconstruction and discrimination pull apart, and only the objective that names the response gets the second one right.

This is why the course trains its feature extractor on the deviance. The next lecture builds exactly that network, and the reason it is worth building rather than pre-training an autoencoder is the 0.644 above.

3 Feed-forward networks generalise the logistic GLM

3.1 A single layer is a scorecard

The architecture the rest of the series depends on can be introduced from where the second lecture stopped. Draw the covariates as a column of input nodes, connect each to a single output node with its own weight, sum, and pass the sum through an inverse link. The result is

$$\boldsymbol{X} \;\mapsto\; \mu_{\vartheta}(\boldsymbol{X}) = g^{-1}\left(\vartheta_0 + \langle \vartheta, \boldsymbol{X} \rangle\right),$$

which is the GLM of the second lecture written as a network. With the logit link and one-hot encoded bands as inputs it is more specific still: it is a scorecard, whose weights are the points awarded per attribute and whose intercept is the base score. A single-layer network is therefore not a new model class for credit at all, and the course's point is the converse one, that a network is the object a scorecard sits inside.

3.2 Depth, and what the layers do

Depth means several transformations applied in sequence. Write $\boldsymbol{z}^{(m)} : \mathbb{R}^{q_{m-1}} \to \mathbb{R}^{q_m}$ for the m-th hidden layer, with q_m neurons, and compose them, $\boldsymbol{z}^{(d:1)} = \boldsymbol{z}^{(d)} \circ \cdots \circ \boldsymbol{z}^{(1)}$. The regression function becomes

$$\boldsymbol{X} \;\mapsto\; \mu_\vartheta(\boldsymbol{X}) = g^{-1}\left(w_0^{(d+1)} + \langle \boldsymbol{w}^{(d+1)}, \boldsymbol{z}^{(d:1)}(\boldsymbol{X})\rangle\right),$$

which is the equation the second lecture closed on. Read from the right, the composition is the feature extractor and the final layer is a GLM on the features it produces. Read from the left, the whole thing is one parametrised family fitted by gradient descent. Both readings are used constantly, and the first is what licenses stripping the last layer off a trained network and feeding its features to something else, e.g. a gradient boosting machine or, as in the section above, a logistic GLM.

Inside each layer the transformation happens in two steps. The **pre-activation** is an affine map, $\boldsymbol{w}_0^{(m)} + W^{(m)}\boldsymbol{x} \in \mathbb{R}^{q_m}$, with weight matrix $W^{(m)} \in \mathbb{R}^{q_m \times q_{m-1}}$ and bias $\boldsymbol{w}_0^{(m)}$. The **post-activation** applies a non-linear function ϕ componentwise,

$$\boldsymbol{x} \;\mapsto\; \boldsymbol{z}^{(m)}(\boldsymbol{x}) = \phi\left(\boldsymbol{w}_0^{(m)} + W^{(m)}\boldsymbol{x}\right) \in \mathbb{R}^{q_m},$$

so every individual neuron is itself a GLM with ϕ as its inverse link. The non-linearity is what makes depth worth having, since a composition of affine maps is affine and a network without activations collapses to the single-layer scorecard above however many layers it has.

The next lecture develops all of this properly, counts the parameters, states the universality theorems and fits the network to the Bondora book. Here it is enough to have the shape.

4 Architectures beyond

the tabular cross-section

4.1 From vectors to tensors

Everything so far treats a borrower as a vector. Credit data often has more structure than that, and the architectures in this section are the ones that respect it. A time series of statements is a matrix $\boldsymbol{X}_{1:t} \in \mathbb{R}^{t \times q}$, an image is a three-dimensional array, and a panel is a set of sequences of differing length.

This series has one dataset of each kind, which is worth setting out plainly because it determines which lectures can demonstrate what. Bondora is a pure cross-section: 45 application-time columns per loan, one row per borrower. The credit card portfolio is also a cross-section, although a peculiar one, since its 664 transaction attributes are aggregates that somebody computed over a history before releasing the file. The Amex book is the genuine article, 5,531,451 customer-month statements across 458,913 customers, up to 13 statements each, right-aligned on a common March 2018 cut-off, with 190 anonymised risk variables per statement. A customer there is a 13×190 tensor.

The credit card file and the Amex panel therefore sit at the two ends of one choice. Either a human decides which windows and which summaries to compute and hands the model a wide cross-section, or the model reads the sequence and learns the summaries. The demo above is a warning about the first route, since 56.8 per cent of that matrix turned out to carry almost no individual default signal, and nobody downstream can tell which aggregation decision cost them what.

4.2 Convolutional networks, and an honest word about credit

A convolutional layer replaces the full connectivity of a network with a window. One filter of kernel size K is applied at every position of the input, so a one-dimensional convolution over a statement history computes

$$z_{u,j}^{(1)} = \phi\left(w_{0,j}^{(1)} + \sum_{k=1}^{K} \langle \boldsymbol{w}_{k,j}^{(1)}, \boldsymbol{X}_{u-1+k} \rangle\right),$$

for filter j at position u. Two properties follow. Local connectivity keeps the parameter count small, and parameter sharing means a pattern is recognised wherever in the window it occurs.

That second property is the one with a credit use. A filter of width three over a delinquency sequence can learn the shape "two months late and then cured" and fire on it whether it happened in month two or month eleven, which is exactly the kind of behavioural pattern a hand-built aggregate such as "maximum delinquency in the last twelve months" flattens away.

The two-dimensional case is thinner here, and it is better to say so than to manufacture a use. Credit has no pixel grid. Geographic default surfaces are the honest exception, and at the granularity a lender actually holds they are usually too sparse to convolve. Telematics heatmaps are an insurance example rather than a credit one. Consequently the convolutional material in this course earns its place in credit through the one-dimensional case and through the vocabulary it gives the attention lectures.

4.3 Recurrent networks and behavioural scoring

A recurrent layer carries a hidden state forward,

$$z_{u,j}^{(1)} = \phi\left(w_{0,j}^{(1)} + \langle \boldsymbol{w}_{j}^{(1)}, \boldsymbol{X}_{u} \rangle + \langle \boldsymbol{v}_{j}^{(1)}, \boldsymbol{z}_{u-1}^{(1)} \rangle\right),$$

so the state at time u summarises everything up to u and the parameters are shared across time. Stated in the language a credit modeller already has, the hidden state is a learned filtration of the account's history, and behavioural scoring is the problem it was built for. Long short-term memory (Hochreiter and Schmidhuber, 1997) and the simpler gated recurrent unit add gates so that a delinquency eleven months ago can still reach the prediction, which plain recurrence struggles with.

The choice between convolution and recurrence follows from what matters in the data. Convolutions suit local patterns and translation invariance and parallelise well;

recurrence suits genuine temporal causality, variable history lengths and long-range dependence. Both are superseded for most purposes by attention, which the transformer lectures take up.

4.4 Embedding layers for categorical covariates

Categorical covariates are where credit and insurance meet most directly. One-hot encoding, which the second lecture used for country, education and employment duration, expresses the prior that the levels are orthogonal, so nothing learned about Estonia informs the estimate for Finland. Actuarial practice has a standard answer to that, namely Bühlmann credibility, which shrinks a level's estimate towards the collective.

An embedding layer is the network's version. Each level is assigned a dense vector of modest dimension, the vectors are trained with the rest of the model, and levels that behave alike end up close together. The payoff grows with cardinality, and credit is full of high-cardinality fields where a one-hot column per level is hopeless: postcode, employer, merchant category, dealer, broker. Richman and Wüthrich (2024) regularise these embeddings by variational inference, which recovers the credibility shrinkage explicitly and is the right reference for anyone who has to defend an embedding to a model risk function.

4.5 Autoencoders, attention and interpretable networks

Three further ideas belong in the overview, and each has its own lecture later.

The **autoencoder** is the unsupervised architecture the demo above used: a network trained to reproduce its own input through a bottleneck, performing a non-linear analogue of principal components without the orthonormality constraint. The bottleneck encodes the prior that the data lie near a low-dimensional manifold. As measured above, that prior is about the covariates rather than about default.

Attention (Vaswani et al., 2017) computes $\mathrm{softmax}(QK^\top/\sqrt{d})\,V$ from query, key and value matrices, letting every element of a sequence attend to every other and producing

contextualised representations. It parallelises where recurrence cannot and reaches arbitrarily far back. The Credibility Transformer lectures build it in full, and the Amex panel is the dataset in this series it was made for.

LocalGLMnet (Richman and Wüthrich, 2023) is the interpretability architecture, and in credit it is arguably the most important idea in the course. A network learns coefficients $w_j(\boldsymbol{X})$ that vary with the covariates, and the prediction is $\sum_j w_j(\boldsymbol{X}) X_j$, so every individual decision decomposes into per-variable contributions. A lender that has to issue reason codes with a declined application needs precisely that decomposition, and getting it from a fitted structure rather than from a post hoc attribution method is a materially stronger position in front of a supervisor.

5 Combined actuarial neural network

5.1 The scorecard the credit committee already approved

The CANN (Wüthrich and Merz, 2019) resolves the tension this lecture opened with. GLMs are interpretable and may miss structure; networks find structure and explain themselves poorly. Rather than choosing, the CANN adds them:

$$\mu^{\text{CANN}}(\boldsymbol{X}) = g^{-1}\Big(\underbrace{\langle \widehat{\vartheta}^{\text{MLE}}, \boldsymbol{X} \rangle}_{\text{GLM baseline}} + \underbrace{\langle \boldsymbol{w}^{(d+1)}, \boldsymbol{z}^{(d:1)}(\boldsymbol{X}) \rangle}_{\text{network correction}} \Big).$$

Training runs in two steps. First the GLM is fitted by maximum likelihood and its parameters are frozen, which for a PD model means the logistic GLM of the second lecture with the logit as canonical link. Then the network is trained to learn what the GLM left on the table, entering the linear predictor as an additive term. He and co-authors (2016) supply the reading that makes the architecture feel inevitable: the GLM term is a skip connection, the network term is the residual branch, and the CANN is a residual network whose skip path happens to be a model somebody has already signed off.

That last clause is why the CANN travels better in credit than almost anything else in the course. A network fitted from scratch replaces an approved model, and replacing an approved PD model means a new development document, a new validation, a new use test and a supervisory conversation. A CANN keeps the approved model as its baseline and adds an increment, so the increment is the thing to be measured, bounded, monitored and, if it misbehaves, switched off. The GLM also keeps predictions sane wherever the network has learned nothing useful, and the size of the correction is itself a diagnostic that says where the scorecard is thin.

5.2 The degenerate CANN is already industry practice

The second credit lecture ended on a piece of standing practice: when a scorecard's ranking survives a shift in the book while its level does not, lenders keep the coefficients and recalibrate the intercept to the current central tendency. In CANN notation that is a correction term restricted to a constant. Reading it that way makes the generalisation obvious, since a constant correction repairs the level everywhere and a learned correction repairs it where it is actually wrong.

One caution carries over from the next lecture. The balance property, i.e. that fitted probabilities sum to observed defaults on the learning sample, holds because the canonical-link GLM's score equations force it. Adding a network term to the linear predictor breaks those equations, so a CANN inherits its baseline's calibration only in the sense that it starts from it. Auto-calibration has to be imposed rather than assumed, which is the subject of the calibration lecture.

Takeaways and outlook

Representation learning is the principle that carries the rest of this series, and this lecture's contribution is to show it is conditional. On a wide anonymised credit card book, a non-linear autoencoder compressed 1,158 covariates to two dimensions and

reconstructed the portfolio 22 per cent better than the optimal linear compression of the same width, while a logistic readout on its codes discriminated default barely better than the base rate. The supervised extractor at the same width reached an AUC of 0.820. The measurement in the last section explains it: the blocks with the most default signal per column hold 8 per cent of the matrix and attract 2.4 per cent of the leading principal component, so an objective that follows the variance follows the wrong thing.

Hence the practical rule for credit. Automated feature learning is worth having, and it has to be trained against the loss the model will be judged by. Every architecture in the remaining lectures does exactly that, and the ones that matter most here (entity embeddings for high-cardinality fields, LocalGLMnet for per-decision reason codes, the CANN for an increment on an approved scorecard) are the ones that buy flexibility without giving up the account of itself a credit model has to be able to give.

The next lecture stops surveying and starts fitting. It builds the feed-forward network in full on the Bondora book, states the universality theorems, works through gradient descent and early stopping, and finds out what a network does to the balance property the second lecture proved.

Copyright and attribution

and the code is Python where the course uses R. The original lecture notes are available at https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5162304.

References

- Altman, E.I. (1968). Financial ratios, discriminant analysis and the prediction of corporate bankruptcy. *Journal of Finance* 23(4), 589-609.
- Baesens, B., Van Gestel, T., Viaene, S., Stepanova, M., Suykens, J. and Vanthienen, J. (2003). Benchmarking state-of-the-art classification algorithms for credit scoring. *Journal of the Operational Research Society* 54(6), 627-635.
- Basel Committee on Banking Supervision (2006). *International Convergence of Capital Measurement and Capital Standards: a Revised Framework, Comprehensive Version*. Bank for International Settlements.
- Bengio, Y., Courville, A. and Vincent, P. (2013). Representation learning: a review and new perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35(8), 1798-1828.
- Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems* 2(4), 303-314.
- Deisenroth, M.P., Faisal, A.A. and Ong, C.S. (2020). *Mathematics for Machine Learning*. Cambridge University Press.
- Djurovic, A. (2025). *Principal Component Analysis for IFRS9 Forward-Looking Modeling*. Model development and validation series, practitioner deck.
- Djurovic, A. (2026). *IFRS9 Forward-Looking Modeling: Supervised Macroeconomic Index*. Model development and validation series, practitioner deck.
- European Banking Authority (2021). *Discussion paper on machine learning for IRB models*. EBA/DP/2021/04, and its 2023 follow-up report.
- European Union (2024). Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence, Annex III.

- Frank, I. and Friedman, J. (1993). A statistical view of some chemometrics regression tools (with discussion). *Technometrics* 35(2), 109-148.
- Hastie, T., Tibshirani, R. and Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd edition. Springer.
- He, K., Zhang, X., Ren, S. and Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 770-778.
- Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation* 9(8), 1735-1780.
- Hornik, K. (1991). Approximation capabilities of multilayer feedforward networks. *Neural Networks* 4(2), 251-257.
- LeCun, Y., Bottou, L., Bengio, Y. and Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE* 86(11), 2278-2324.
- Lessmann, S., Baesens, B., Seow, H.-V. and Thomas, L.C. (2015). Benchmarking state-of-the-art classification algorithms for credit scoring: an update of research. *European Journal of Operational Research* 247(1), 124-136.
- Richman, R. and Wüthrich, M.V. (2023). LocalGLMnet: interpretable deep learning for tabular data. *Scandinavian Actuarial Journal* 2023(1), 71-95.
- Richman, R. and Wüthrich, M.V. (2024). High-cardinality categorical covariates in network regressions. *Japanese Journal of Statistics and Data Science* 7(2), 921-965.
- Stone, M. and Brooks, R.J. (1990). Continuum regression: cross-validated sequentially constructed prediction embracing ordinary least squares, partial least squares and principal components regression. *Journal of the Royal Statistical Society, Series B* 52(2), 237-269.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L. and Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems* 30.
- Wiginton, J.C. (1980). A note on the comparison of logit and discriminant models of consumer credit behavior. *Journal of Financial and Quantitative Analysis* 15(3), 757-770.
- Wold, H. (1975). Soft modelling by latent variables: the nonlinear iterative partial least squares (NIPALS) approach. In *Perspectives in Probability and Statistics, In Honor of M.S. Bartlett*, 117-144.

- Wüthrich, M.V. and Merz, M. (2019). Editorial: Yes, we CANN! *ASTIN Bulletin* 49(1), 1-3.
- Wüthrich, M.V. and Merz, M. (2023). *Statistical Foundations of Actuarial Learning and its Applications*. Springer.
- Zeiler, M.D. and Fergus, R. (2014). Visualizing and understanding convolutional networks. *Computer Vision - ECCV 2014*, 818-833. Springer.
- Scognamiglio, S., Maggi, M. and Wüthrich, M.V. (2026). *Deep Learning for Actuarial Modeling*, SSRN 5162304.